34 NP 完備性 解答

34-E01 NTM witness 與 verifier 的形式對應

推導

NTM 每一條 path 先猜一個長度 <=q(|x|) 的 y，再 deterministic 執行 V(x,y)。單條 path 是多項式長；不是以 deterministic polynomial time 列舉所有 y，那通常有指數多個。若 x in L，completeness 給一條接受 path；若 x notin L，soundness 使所有 path 都拒絕。

結論與檢核

反向給定 polynomial-time NTM M。把某條 accepting computation path 的 nondeterministic choices 寫成 certificate y。Choice 數受 polynomial time bound 限制，故 |y| polynomial；deterministic verifier 模擬 M 沿該 path 的 transition。M 接受若且唯若存在這串 choices 使模擬接受。兩方向共同得到 verifier 定義與 NTM 定義等價；量詞結構是對每個x都有「x∈L iff 存在有界長度y使V接受」；不能寫成所有x都存在接受證書，因no instance沒有接受證書。

34-E02 SUBSET-SUM 的 bit length 與 pseudopolynomial

推導

每個 a_i 為 $\Theta(m)$ bits，t 有 $m+1$ bits，故 $N=\Theta(m^2)$。標準完整表 DP 成本為 $\Theta(m2^m)=2^{\Theta(\sqrt{N})}$。若存在固定 c,C 使 $m2^m\le CN^c=O(m^{2c})$，則指數 2^m 被固定次多項式上界，矛盾。此結論談的是這個 DP 的 worst-case/table 工作量，不是從一個 O 上界單獨推出任何演算法都需指數時間。

結論與檢核

固定 $n=200, B=2^{80}$ 只是單一有限 instance，無法證明漸近階。此參數族各 $a_i=\Theta(2^m)$，全 unary 輸入長度 $\Theta(m2^m)$，本 DP 成本便是輸入長度多項式。一般 unary 只需使用輸入長至少含 $n+t$，$nt\le((n+t)/2)^2$ 即足夠，不可把每數僅有上界 B 誤寫成一律 $\Theta(nB)$ 長度。

34-E03 完整 3-CNF-SAT 到 CLIQUE 構造

推導

三群 C1={x1,¬x2,x3}、C2={¬x1,x2,¬x3}、C3={x1,x2,x3}，群內無邊，跨群非互補才有邊。取 x1=F、x2=F、x3=T，三 clauses 都真。選 C1 的 x3、C2 的 ¬x1、C3 的 x3，三 literal 均為真且彼此不互補，得 3-clique。原先 x1=T、x2=F、x3=T 會使第二 clause 全假，不能當 satisfying witness。

34 NP 完備性 解答 (續)

結論與檢核

Completeness：satisfying assignment 下每群選一個 true literal，因一致 assignment，選到的一對不會是 x 與 ¬x，故 k-clique。Soundness：同一群不連，所以 3-clique 每群恰選一點。任意被選 literals 均有跨群 edge，故不存在 complement pair；賦予被選 literal 為真，其他變數任意，則每群至少一 literal 真，F satisfiable。頂點數3k，檢查所有跨群 pair需$O(k^2)$次word比較。先把變數名稱編成n個ID，bit計費可採$O(|F| \log|F|+k^2 \log(n+k))$上界，仍是輸入長度多項式；iff因而完成。

34-E04 CLIQUE 與 VERTEX-COVER 的雙向轉換

推導

G 中 {1,2} 是 2-clique。補圖 Gbar 的唯一 edge 是 13；在 Gbar 中 {1,2} 沒有邊，是 independent set。其補集 {3} 覆蓋 Gbar 的唯一 edge，所以 size 為 n-k=1 的 vertex cover。

一般證明：若 C 是 G 的 clique，對任意 u,v in C，G 有 edge，所以 Gbar 沒有相應 edge，C 是 independent set。若 S 在 Gbar independent，任何 Gbar edge 不可能兩端都在 S，因此 V\S 覆蓋全部 Gbar edges。反向若 C0 覆蓋 Gbar，則 V\C0 不可能含一條 Gbar edge，故 V\C0 是 Gbar 的 independent set，也就是 G 的 clique。

結論與檢核

由 c 是 size-k clique 等同其補集是 size-(n-k) vertex cover，得 `(G,k)->(Gbar,n-k)`；把 H=Gbar 套用同一命題，得反向 `(H,1)->(Hbar,n-1)`。兩者都只建立 O(n^2) 條補邊，故是雙向 polynomial-time many-one reduction，也解釋 `k` 與 `n-k` 的角色。

34-E05 Hamiltonian cycle 到 TSP decision

推導

三角形 K3 三條原圖 edges 權重 1，令 B=3。候選 tour 1-2-3-1 成本 3，等於 B。一般 G 建 complete graph K_n，G edges 權 1、非 edges 權 2，B=n。HC 的 n 條邊形成成本 n tour。反向任一 tour 恰有 n 條，每條至少權 1；若總成本 <=n，全部必須權 1，故所有 tour edges 屬於 G，tour 是 HC。

34 NP 完備性 解答 （續）

結論與檢核

因此 `HC(G) iff exists tour of cost<=n`。TSP decision 的 certificate 是 n 個頂點的 permutation，verifier 檢查每點一次、首尾相接、所有邊存在於 complete graph、總成本 <=B，全為 polynomial。Optimization TSP 回傳最小成本與 tour；decision 只回答門檻是否存在，不能把 NP-complete 的 decision 問題與要求輸出完整最佳 tour 的 optimization 輸出型別混為同一語言。

34-E06 many-one reduction 的傳遞性與方向

推導

定義 h(x)=g(f(x))。若 x in A，f 的 iff 給 f(x) in B，再給 g(f(x)) in C。若 h(x) in C，g 的 sound direction 給 f(x) in B，再由 f 的 sound direction 得 x in A。兩向均有 `x in A iff h(x) in C`。時間：f 在 p_f(|x|)，其輸出長度也受 polynomial 限制，g 在 p_g(|f(x)|)，總時間 `O(p_f(n)+p_g(p_f(n)))`，是多項式。

結論與檢核

故 many-one reduction 有傳遞性。若已知 C 是 NP-hard 且 `C<=_p L`，任取 A in NP，先 `A<=_p C` 再接到 L，才得 L NP-hard。若只知 `L<=_p C`，hardness 從 C 不會逆流到 L；它只表示 L 不比 C 難。最後還要另證 `L in NP`，才可把 NP-hard 升級為 NP-complete。

34-E07 Gate 約束不能只保留一個方向

推導

OR gate給(u∨¬a)∧(u∨¬b)∧(¬u∨a∨b)；NOT給(v∨a)∧(¬v∨¬a)；AND給(¬z∨u)∧(¬z∨v)∧(z∨¬u∨¬v)，另加unit clause z。取a=0,b=1，沿電路得到u=v=z=1，逐式皆真。

若此AND的z已被強制1，漏掉(z∨¬u∨¬v)不會在這個局部造成錯誤，因該式本來就自動為真；剩下兩式仍迫使u=v=1。但這不代表省略對所有電路安全。

反例為z=a∧b、w=¬z，另外把a,b固定為1並要求w=1。真正電路只能z=1,w=0，無解；若AND只保留z→a、z→b，則可偽造a=b=1,z=0,w=1，使所有不完整約束與NOT約束都成立。缺少u∧v→z正好允許gate輸出假造為0。

結論與檢核

Gate替換需等價的雙向約束，不能由某個輸出恰被固定的特例推出一般省略合法。沿拓樸序恢復真wire值的證明，依賴每個gate都完整。

34 NP 完備性 解答 （續）

34-E08 HC Gadget 的局部與全域

推導

a2,a5,b2,b5各度2，迫使八條鄰接鏈邊。a3,a4分別已有一條強制邊，所以a3b1=a4b6=1−x；b3,b4同理使a1b3=a6b4=1−y。a1,a6內度2−y、外度y；b1,b6內度2−x、外度x。

x=y=0時有cycle a1−a2−a3−b1−b2−b3−a1，及a4−a5−a6−b4−b5−b6−a4。所有點已度2、無外接端口，無法連到任何selector，不能成為全圖單一HC。

取U=W_uv（a側u、b側v）、V=W_vw（a側v、b側w）。完整cycle為：

段落	依走訪順序的十二點路徑
U	b1 → b2 → b3 → a1 → a2 → a3 → a4 → a5 → a6 → b4 → b5 → b6
V	a1 → a2 → a3 → b1 → b2 → b3 → b4 → b5 → b6 → a4 → a5 → a6

每行點名都加該行gadget前綴。由s1進U.b1，完整走U行，經U.b6→V.a1串接；完整走V行後，由V.a6回s1。這明列全部24個gadget頂點及唯一selector，保留首尾閉合。

U中採x=1,y=0，V中採x=0,y=1；兩段分別為v側完整路徑。U.b6–V.a1是原頂點v的串鏈，首尾兩邊接s1。U/V各12點，加s1共25個唯一頂點。

結論與檢核

每個gadget選到的一側必整對端口外接，不能走入u側卻改由v側外鏈離開。局部模式與全域selector／串鏈共同保證能解回cover；只檢查局部圖有路徑還不足以證全域歸約。

34-E09 無進位歸約與反向解碼

推導

欄序x1,x2,x3,C1,C2：T1=10010、F1=10001；T2=01001、F2=01010；T3=00111、F3=00100；slack為00010、00020、00001、00002，target=11144。

assignment(0,1,1)選F1+T2+T3=10001+1001+111=11113，clause真數為1、3。選S1+S1'+S2=10+20+1，總11144。assignment(0,1,0)選F1+T2+F3=11102，第一clause真數0，兩個第一欄slack最多3，不能到4；其他欄無進位，也不能補足此缺口。

對任意子集合，變數欄最多兩個1，clause欄最多來自三個literal的1加slack的1與2，和≤6<10。故每個欄的總和直接獨立對應target。達target時變數欄必恰選T_i/F_i之一，clause欄扣掉最多3的slack後至少有一個真literal。

34 NP 完備性 解答 （續）

結論與檢核

一般n變數k個clauses，2n+2k個數各有n+k個十進位欄、O(n+k)個binary bits，總 $O((n+k)^2)$ bits。數值可能指數大，卻不影響多項式輸出長度；對所有候選無進位，才支持反向解碼。

原創驗算程式附錄: clique_reduction.py

```
"""Check both directions of the Chapter 34 3-CNF-SAT to CLIQUE reduction."""

from itertools import combinations, product

# Positive integer means x_i; negative integer means not x_i.
formula = (
    (1, -2, 3),
    (-1, 2, -3),
    (1, 2, 3),
)


def satisfiable(clauses):
    variables = sorted({abs(literal) for clause in clauses for literal in clause})
    for bits in product((False, True), repeat=len(variables)):
        assignment = dict(zip(variables, bits))
        if all(any(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in
clauses):
            return True
    return False


def build_graph(clauses):
    vertices = [(row, column, literal) for row, clause in enumerate(clauses) for column,
literal in enumerate(clause)]
    edges = set()
    for left, right in combinations(vertices, 2):
        if left[0] == right[0]:
            continue
        if left[2] == -right[2]:
            continue
        edges.add(tuple(sorted(((left[0], left[1]), (right[0], right[1])))))
    return vertices, edges


def has_k_clique(vertices, edges, k):
```

34 NP 完備性 解答 （續）

```
    for selected in combinations(vertices, k):
        if len({vertex[0] for vertex in selected}) != k:
            continue
        pairs = combinations(selected, 2)
        if all(tuple(sorted(((a[0], a[1]), (b[0], b[1])))) in edges for a, b in pairs):
            return True
    return False


def verify_equivalence(clauses):
    vertices, edges = build_graph(clauses)
    return satisfiable(clauses) == has_k_clique(vertices, edges, len(clauses))


clause_domain = tuple(product((1, -1, 2, -2), repeat=3))
examples = (
    formula,
    ((4, 4, 4), (-4, -4, -4)),
    (),
) + tuple(
    (left, right)
    for left in clause_domain
    for right in clause_domain
)
for example in examples:
    assert verify_equivalence(example), example

vertices, edges = build_graph(formula)
print(
    f"cases={len(examples)} vertices={len(vertices)} edges={len(edges)} "
    "satisfiable_iff_k_clique=verified"
)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性 解答 （續）

原創驗算程式附錄: reduction_constructions.py

```
"""Executable checks for the explicit Boolean, HC, and subset-sum gadgets."""
from itertools import combinations, product


def eval_clause(clause, assignment):
    return any(assignment[abs(lit)] == (lit > 0) for lit in clause)


def gate_clauses(kind, z, u, v=None):
    if kind == 'and':
        return ((-z,u),(-z,v),(z,-u,-v))
    if kind == 'or':
        return ((z,-u),(z,-v),(-z,u,v))
    return ((z,u),(-z,-u))


def pad(clause, p, q):
    if len(clause)==3:
        return (clause,)
    if len(clause)==2:
        return (clause+(p,),clause+(-p,))
    return tuple(clause+(a*p,b*q) for a,b in product((1,-1),repeat=2))


def widget_edges():
    return ([(i,i+1) for i in range(5)]
            +[(i,i+1) for i in range(6,11)]
            +[(0,8),(2,6),(3,11),(5,9)])


def widget_modes():
    edges=widget_edges()
    ports={0,5,6,11}
    modes=[]
    for mask in range(1<<len(edges)):
        adjacency=[[] for _ in range(12)]
        for i,(u,v) in enumerate(edges):
            if mask>>i&1:
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性 解答 （續）

```
                adjacency[u].append(v)
                adjacency[v].append(u)
        if any(len(adjacency[v])!=2 for v in range(12) if v not in ports):
            continue
        if any(len(adjacency[v])>2 for v in ports):
            continue
        seen=set()
        endpoints=[]
        valid=True
        for start in range(12):
            if start in seen:
                continue
            todo=[start]
            component=[]
            while todo:
                u=todo.pop()
                if u in seen:
                    continue
                seen.add(u)
                component.append(u)
                todo.extend(adjacency[u])
            ends=[u for u in component if len(adjacency[u])<2]
            if len(component)==1 and start in ports:
                endpoints.append((start,start))
            elif len(ends)==2 and set(ends)<=ports:
                endpoints.append(tuple(sorted(ends)))
            else:
                valid=False
                break
        if valid:
            modes.append(tuple(sorted(endpoints)))
    return sorted(modes)


def numbers_for(clauses, n):
    k=len(clauses)
    def encode(digits):
        return int(''.join(map(str,digits)))
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性 解答 （續）

```
    values=[]
    for i in range(1,n+1):
        for literal in (i,-i):
            digits=[int(j==i-1) for j in range(n)]
            digits += [int(literal in clause) for clause in clauses]
            values.append(encode(digits))
    for j in range(k):
        for digit in (1,2):
            values.append(encode([0]*n+[digit if r==j else 0 for r in range(k)]))
    target=encode([1]*n+[4]*k)
    return values,target


def verify():
    gate_cases=0
    for kind in ('and','or','not'):
        for bits in product((False,True),repeat=3):
            assignment=dict(zip((1,2,3),bits))
            expected=bits[0] and bits[1] if kind=='and' else bits[0] or bits[1]
            if kind=='not':
                expected=not bits[0]
            result=all(eval_clause(c,assignment) for c in gate_clauses(kind,3,1,2))
            assert result==(bits[2]==expected)
            gate_cases+=1
    padding_cases=0
    for clause in ((1,),(1,2)):
        for bits in product((False,True),repeat=2):
            original=dict(zip((1,2),bits))
            extensions=[]
            for extras in product((False,True),repeat=2):
                assignment=original|dict(zip((3,4),extras))
                extensions.append(all(eval_clause(c,assignment) for c in pad(clause,3,4)))
            assert any(extensions)==eval_clause(clause,original)
            padding_cases+=1
    modes=widget_modes()
    assert modes==[((0,5),),((0,5),(6,11)),((6,11),)],modes
    clauses=((1,2,3),(-1,2,-3))
    values,target=numbers_for(clauses,3)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性 解答 （續）

```
    assert values==[10010,10001,1011,1000,110,101,10,20,1,2]
    assert target==11144
    chosen=[values[i] for i in (0,2,5,7,9)]
    assert sum(chosen)==target
    subset_cases=0
    satisfiable_masks=[]
    for mask in range(1<<len(values)):
        selected=[v for i,v in enumerate(values) if mask>>i&1]
        if sum(selected)!=target:
            continue
        assignment={}
        for i in range(3):
            assert ((mask>>(2*i))&1)+((mask>>(2*i+1))&1)==1
            assignment[i+1]=bool(mask>>(2*i)&1)
        assert all(eval_clause(c,assignment) for c in clauses)
        subset_cases+=1
        satisfiable_masks.append(mask)
    assert subset_cases>0
    for bits in product((False,True),repeat=3):
        assignment=dict(zip((1,2,3),bits))
        if not all(eval_clause(c,assignment) for c in clauses):
            continue
        base=sum(values[2*i+(not bits[i])] for i in range(3))
        slack=0
        for j,c in enumerate(clauses):
            count=sum(assignment[abs(lit)]==(lit>0) for lit in c)
            slack+=(4-count)*10**(len(clauses)-j-1)
        assert base+slack==target
    print(f'PASS gate_truth_rows={gate_cases} padding_assignments={padding_cases} '
          f'widget_edge_subsets={1<<14} widget_modes={len(modes)} '
          f'subset_masks={1<<len(values)} valid_subsets={subset_cases}')


if __name__=='__main__':
    verify()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

34 NP 完備性 解答 （續）

原創驗算程式附錄: reduction_depth.py

```
"""Witness construction for the chapter's concrete VC-to-HC reduction."""
from itertools import combinations
from pathlib import Path
import sys
sys.path.insert(0,str(Path(__file__).resolve().parent))
from reduction_constructions import widget_edges,numbers_for,gate_clauses,eval_clause

def cover_to_cycle(vertices,edges,cover):
    """Nonisolated simple graph, nonempty valid cover; returns graph and HC witness.

    This builds the witness from a supplied cover; it does not solve HC.
    """
    vertices=sorted(vertices);edges=sorted(tuple(sorted(e)) for e in
edges);cover=sorted(cover)
    if not cover or not all(u in cover or v in cover for u,v in edges):raise
ValueError('nonempty valid cover required')
    neighbors={u:sorted(v if u==w else w for w,v in edges if u in (w,v)) for u in vertices}
    if any(not neighbors[u] for u in vertices):raise ValueError('remove isolated vertices
first')
    graph={}
    def add(u,v):graph.setdefault(u,set()).add(v);graph.setdefault(v,set()).add(u)
    def node(u,v,i):
        a,b=sorted((u,v));return (a,b,i if u==a else i+6)
    for u,v in edges:
        for i,j in widget_edges():add((u,v,i),(u,v,j))
    for u in vertices:
        for a,b in zip(neighbors[u],neighbors[u][1:]):add(node(u,a,5),node(u,b,0))
        for j in range(len(cover)):
            add(('s',j),node(u,neighbors[u][0],0));add(('s',j),node(u,neighbors[u][-1],5))
    cycle=[]
    for j,u in enumerate(cover):
        cycle.append(('s',j))
        for v in neighbors[u]:
            a,b=sorted((u,v))
            if v in cover:indices=list(range(6)) if u==a else list(range(6,12))
            elif u==a:indices=[0,1,2,6,7,8,9,10,11,3,4,5]
            else:indices=[6,7,8,0,1,2,3,4,5,9,10,11]
            cycle.extend((a,b,i) for i in indices)
    return graph,cycle

def valid_cycle(graph,cycle):
    return len(cycle)==len(set(cycle))==len(graph) and set(cycle)==set(graph) and all(v in
graph[u] for u,v in zip(cycle,cycle[1:]+cycle[:1]))

if __name__=='__main__':
    g,c=cover_to_cycle([0,1,2],[(0,1),(1,2)],[1])
    print('vertices',len(g),'cycle',c,'valid',valid_cycle(g,c))
```

34 NP 完備性 解答 （續）

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study